Lower bound study for the concurrent multicast from multiple sources

by
Wen-Lin Yang
Department of Information Technology
National Ping-Tung Institute of Commerce
No. 51, Ming-Sheng East Road, Ping-Tung, Taiwan
Email: wly@npic.edu.tw

Abstract

For some real-time applications on the network, like videoconference and on-line game, a group of multicast routing trees may exist simultaneously. In this paper, we study the problem that concerns about how to accommodate multiple multicast trees on the network without violating bandwidth constraints. In order to estimate the performance of heuristic algorithms, we present a method to establish the lower bound for the NP-complete problem. In addition, two heuristic methods, modified-MMST and tabu-search based procedures, are proposed to search for the approximation solutions for the problem. Based on the experiments conducted in this study, the approximation solutions found by modified-MMST method and tabu-search based heuristic are very close to the optimal solutions for small-scale networks. The relative error is not greater than 0.78%. For large-scale networks, the experimental results show that the tabu-search based heuristic is an effective procedure for the problem.

Keyword: tabu search, Steiner tree problem, group multicast, routing

1. Introduction

Multicast communication is a transmission mechanism that can send messages from one source node to several destinations, if the source and destinations are on the same multicast tree. Since a message package can be duplicated at the intermediate nodes, only one copy of message is sent from the source node to the destinations. This mechanism saves a large amount of bandwidths especially when the transmitted data are multimedia streams. Therefore, modern multimedia applications, like video-on demand systems [12-14], are mostly built based on multicast communication.

The traditional multicast is a one-to-many communication mechanism. However, in some multimedia applications, many-to-many multicasting is needed. For example, in some applications like videoconferences or on-line games, every member node in a group may want to transmit messages to the other member nodes concurrently. That is, every member node can act as a multicast source as well as a multicast destination. Hence, a multicast routing tree is needed for every member node participating the concurrent group multicasting. The problem, that concerns how to accommodate multiple multicast routing trees on a network such that the bandwidth constraints are satisfied and the total cost of trees is minimized, is often referred as the Group Multicast Routing problem. It is called the GMR problem in this paper.

In case that only one source node in the network and all the links own infinite amount of bandwidth, our GMR problem is reduced to the famous minimum Steiner tree (MST) problem, which is NP-complete. A MST problem is to find a tree spanning a given subset of nodes in a graph such that the total cost of all the links of the tree is minimized. A number of excellent heuristic algorithms were proposed in the past [1-4]. Hence, no feasible solutions in polynomial time exist for our GMR problem. As we know, there are only two heuristic methods developed for this problem [9,10]. In paper [9], by taking bandwidth constraints into considerations, their approach constructs multicast trees using the KMB heuristic [4] for every source node. During the construction, some edges (links) may be saturated because of the number of multicast trees utilizing a link is over the amount of bandwidth assigned to the link. Based on the principle of minimizing the overall cost of the set of multicast routing trees, some trees are selected and rebuilt without including those saturated edges. A similar approach is also proposed in paper [10], but the least-cost set of multicast trees is computed using the TM heuristic [3], due to the empirical studies [11] show that the TM heuristic is superior to KMB heuristic.

In this paper, we present a heuristic for the GMR problem based on the tabu search method [15], which has been proved to a very successful technique for the nonlinear optimization problems [6,7,8]. Since the exact solutions with minimum cost of the problem cannot be obtained without enumerating all the combinations of links, it is infeasible to compute the exact optimal solutions even for a small-scale network. Hence, it would be an advantage to establish a lower bound to approximately estimate the performance of the heuristic algorithms. Based on the multiple minimum spanning trees, a method called MMST is developed in section 3 to compute the lower bound for the GMR problem. A heuristic modified from MMST procedure and named modified-MMST procedure is also presented. The modified-MMST procedure works better for small-scale networks, while tabu-based heuristic is an efficient procedure for large-scale networks. A performance comparison between lower bound, modified-MMST method and tabu-based method is presented in section 5 as well.

2. Problem specification for the concurrent multicast from multiple sources

The network studied in this paper can be modeled by a directed graph $G(V, E)$, where V is the set of communication nodes and E is the set of communication links. Each link $(i, j) \in E$ is a directed edge from node i to node j, and it has two parameters: link cost $c_{i,j}$ and link bandwidth $b_{i,j}$. However, $b_{i,j}$ may not be equal to $b_{i,j}$, and the values of $c_{i,j}$ and $c_{j,i}$ could be different. For example, in Fig. 1, each link has a pair of integers. The first integer represents the link cost and the second one represents the available bandwidth of the link.

Let D be a subset of V ($D \subset V$ and $|D| = m$), and denote the source group. A node in D is called a source node. A source node i transmits multicast streams to every member node j in D, where $i \neq j$. Hence, for every member node in D, there is a multicast routing tree rooted at it.

Our group multicast routing problem is to find a set F that contains m multicasting trees. Let $F = \{T_0, T_1, ..., T_{m-1}\}$, each routing tree T_i rooted at node i must cover all the member nodes in D. We also assume that a multicasting stream flowing on a link requires one unit of bandwidth. A feasible solution F must satisfy the following constraint: the number of streams flowing on any link cannot exceed the maximum bandwidth available on the link. Therefore, our GMR problem can be formulated as the following optimization problem:

$$\text{Minimize: } \sum_{k=0}^{m-1} \cos t(T_k), \text{ where } \cos t(T_k) = \sum c_{i,j}, \quad (i, j) \in T_k \text{ and } i, j \in V. \qquad \text{[A]}$$

Subject to: T_k covers all the member nodes in D and the total bandwidth allocated from a link cannot exceed its available bandwidth.

If we ignore the bandwidth constraints imposed on all the links, then the above optimization problem is reduced to a multiple minimum Steiner tree problem whose goal is to find a group of minimum Steiner trees rooted at all member nodes respectively. In the GMR problem, how to accommodate multiple multicast routing trees on the network in order to satisfy the bandwidth constraints imposed on all the links is the goal of this study. For example, based on the network given in Fig. 1, there are three distinct routing trees rooted at three different source nodes, where all the nodes are assumed to be in the same multicasting group. Two sets of the multicast routing trees are given in Fig. 2 and Fig. 3, where the overall costs of the routing trees are 10 and 11 respectively.

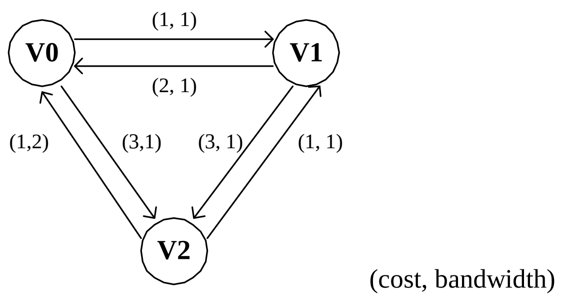

Figure 1. A simple network modeled by a directed graph.

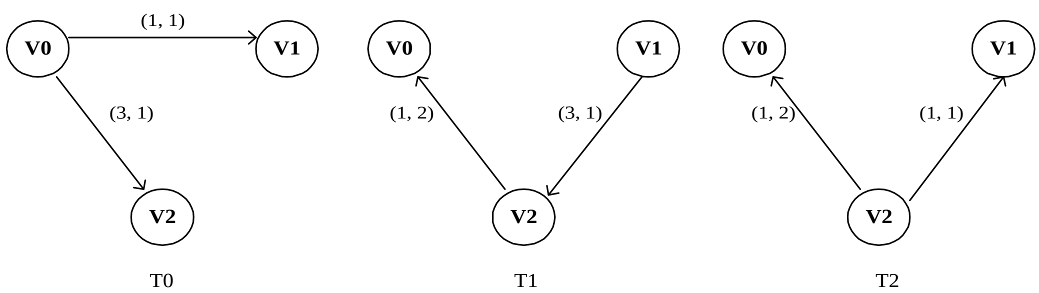

Figure 2. The first set of group multicast routing trees

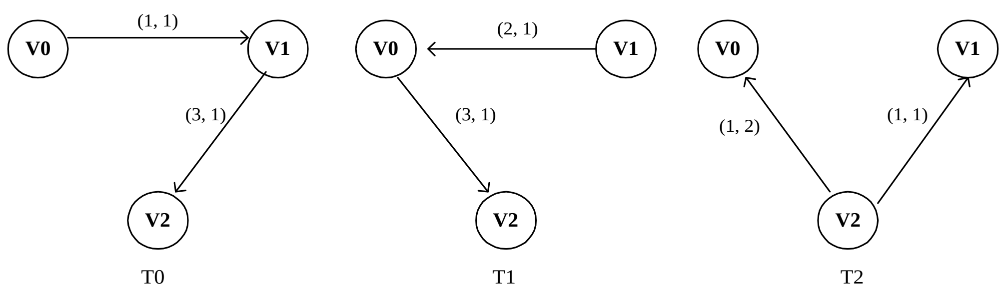

Figure 3. The second set of group multicast routing trees

3. Lower bound study, MMST and modified-MMST methods

Since the GMR problem is a NP-complete problem and the minimal-cost solutions cannot be determined without taking all the links into considerations, it is very difficult to find the exact solutions for a small-scale network. The reason is that for a network with around 10 nodes the number of links could be much larger than 30, and the number of different combinations of links needed to take into considerations is so huge that it is impossible to obtain the optimal solutions with reasonable time and space. Hence, a proper exhaustive procedure for finding the exact optimal solutions should not be developed based on enumeration of all links. In case that exact solutions cannot be easily determined, we should find a way to establish a lower bound for the optimal solutions. With a proper lower bound, it would be helpful for us to evaluate the quality of approximate solutions found by heuristic algorithms.

In this section, based on multiple minimal spanning trees, we present a procedure to establish a lower bound for the optimal solutions of GMR problem. This procedure is named MMST procedure in this paper. The lower bound could be less than the value of an exact optimal solution, because it is computed based on a set of minimum spanning trees which is determined by assuming each link with infinite large of bandwidth.

With the same network model described in the previous section, the basic idea of the MMST method is that we first enumerate all the subsets of the nodes in the network, and keep the subsets that can cover all the source nodes. For each subset, the minimal spanning trees are then computed for each source node. There are a number of minimal spanning trees found for each source node. The least-cost minimal spanning tree of each source node is then selected to form a candidate set. We let R denote the candidate set. The overall cost of set R is the lower bound for our GMR problem. However, some links in set R could be overloaded. In the case that no overloaded link is found in set R, the candidate set is an optimal solution of the problem. This algorithm is listed in the Figure 4.

In the case that overloaded links contained in set R, we propose a method to find an approximation solution, which is believed to be very close to optimal solutions. This method is based on the following observations: although the set R is not a valid solution set of GMR problem, the valid optimal solutions should be very similar to the set R. It is unlikely that in the optimal solutions a valid minimal multicasting tree based on a source node is very different from all the minimal spanning trees rooted at the node, where the minimal spanning trees are found using the method described in the Figure 4. Therefore, it would be helpful if we take the neighbors of a minimal spanning tree into considerations when an approximation solution is determined.

In this paper, the neighbor trees of a minimal spanning tree are defined by replacing a tree edge with an edge that is not contained in the original minimal spanning tree. In Figure 5, for example, the trees in (b) and (c) are the neighbor trees of the minimal spanning tree shown in (a). However, not all neighbors are valid. A valid neighbor tree must cover all the source nodes. The neighbor trees in Figure 5 are called first order neighbors, because they are obtained by one-edge replacement of the original minimal spanning tree. With the similar replacement strategy, the mth order neighbors can be recursively derived from the $(m-1)th$ order neighbors. As a result, for each source node, we may append all the distinct neighbors to their corresponding minimal spanning trees to form a spanning-tree list. An approximation solution can be found by the following procedure, which is named modified-MMST procedure:

a. The spanning-tree lists of all source nodes are sorted into increasing order separately.
b. Starting from the least-cost spanning trees on the sorted lists, we may find a set of spanning trees with the minimal total cost. This set of spanning trees must contain only one tree from one sorted list. In addition, no link in the set is overloaded. This set of spanning trees is an approximation solution for the GMR problem.

Since the number of spanning trees of each source node is exponential proportion to the number of nodes in the network, the procedure described above is an exponential time algorithm. Although this heuristic algorithm is only good for small-scale network, the approximation solutions found with this method is very close to lower bound according to the simulation results of Table 1 and 2 shown in section 5. It implies that the approximation solution found by modified-MMST procedure is very close to the optimal solutions.

1. Given a network $G(V, E)$, where if a link $(i, j) \in E$, then there exits a link $(j, i) \in E$. There are two parameters: cost and bandwidth, on every link.
2. Let D is a set of all source nodes, $D \subset V$.
   Generate a set F, where $F = \{W \mid D \subset W, W \subset V\}$.
3. For each node-set W in F {
   Construct a sub-network $G^{'}(W, E^{'})$, $E^{'} = \{(u, v) \mid u \in W, v \in W, (u, v) \in E\}$.
   Test if $G^{'}$ is a connected digraph starting from the source nodes in D;
   If it is, for each source node u in D, we generate the corresponding minimum spanning tree rooted at u.
   }
4. Let $R = \{T \mid T \textit{ is the least} - \cos t \textit{ MST rooted at } u, \forall u \in D\}$,
   Lower bound $B = \sum_{i=0}^{m-1} \cos t(T_i), T_i \in R$

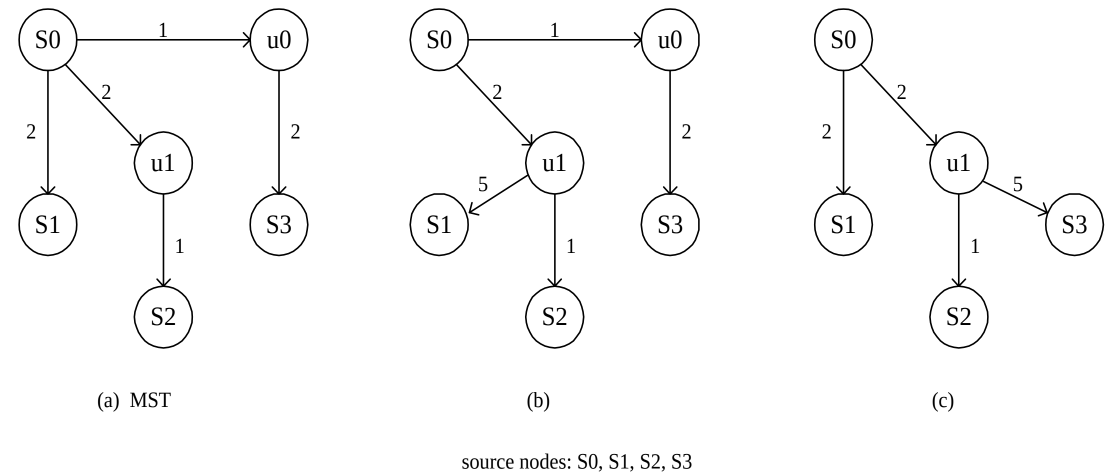

Figure 5 A minimal spanning tree and its first-order neighbor trees.

4. Our tabu-search based heuristic algorithm

Our tabu-search based heuristic algorithm begins with an initial solution, which is found by a greedy procedure described in the following subsection. New sets of solutions are then iteratively generated based on the tabu-search strategy. The tabu-search based heuristic procedure stops if solution quality does not be improved after a number of iterations.

4.1. A greedy procedure

In this subsection, a simple greedy procedure as shown in Figure 6 is developed. This method begins to generate the Steiner trees from the original network one by one without taking the bandwidth constraints into considerations. A popular heuristic algorithm called TM heuristic [3] is applied to generate these approximate Steiner trees. The TM heuristic begins with an arbitrary node in the multicast destination set, for example the source, and repeatedly adds the remaining members to the tree via shortest paths in order of closest first. For our problem, a number of links included in the Steiner trees may be overloaded. For an overloaded link e, a repair procedure, included in greedy procedure, is called to rebuild the Steiner tree without the link e using the TM heuristic. The repair procedure stops when the utilization of link e is not greater than its allocated bandwidth.

For some networks, this approach may fail to find a feasible solution without violating the bandwidth constraints; even at least one feasible solution exists. On the other hand, a feasible solution found by the greedy method may not be the optimal solution for the problem. For example, in Figure 2 and 3, two different configurations of the group multicast trees are due to two different ways used for allocating the links in T_0.

4.2. Tabu search based algorithm

In this subsection, based on tabu-search strategy, we introduce a new heuristic algorithm for the group multicast routing problem. Our method begins with a valid initial solution found by the greedy method. Our tabu search based procedure can iteratively reduce the overall link cost for the current solution. For an iteration loop, a link is randomly selected and temporarily removed from the multicast trees that contain this link. These multicast trees become graphs with two disconnected sub-trees. For each graph, the sub-tree with the source node is kept and the other sub-tree is deleted. Based on the sub-tree with the source node, a TM procedure is used to rebuild a multicast tree, which can cover all the other source nodes that are not included in this sub-tree. For each new generated multicast tree, the tree cost is computed. Let U denote the set of these new generated multicast trees. For each tree $T_k^{'}$ in U, we define the overhead H = cost of $T_k^{'}$ - cost of T_k, where T_k is the original multicast tree rooted at node k. A tree $T_k^{'}$ in U with the smallest value of H is selected to replace T_k to be the new multicast tree for node k. The other trees in U are discarded. As a result, a new solution is generated and accepted to be the current solution for next iteration loop; even the overall cost is higher than previous one.

The above procedure is a local search procedure based on one selected link. A selected link is then pushed into a queue. This queue is a tabu-list. For each iteration loop, a link in queue cannot be selected until it leaves the tabu-lists. In some cases, the selected link cannot be removed from any multicast tree, and no multicast tree can be constructed without it. That is, it is a critical link. Another tabu-list is maintained for storing the critical links. A critical link cannot be selected during the following iterations. The best solution is kept as the iteration proceeds.

By keep accepting the new solutions even they have higher tree costs than previous one, it may lead the searching paths running into a wrong direction. Therefore, after several iteration loops without generating a better solution, we restart the searching process using the best solution found so far to be a new current solution. The procedure stops after a number of iterations without generating a better solution. Our algorithm is outlined in Figure 7. As for the time complexity, the running time is not a fix value, because it depends on the number of times of line [1] in Figure 7 being executed. In Figure 7, line [1] is executed when the global best solution is improved, whereas the number of times of improvement is determined by the benchmarks and simulation conditions.

Greedy procedure () {
Let $F = \{T_0, T_1, ..., T_{m-1}\}$, and T_i is a multicast routing tree;
Assume that each link owns infinite bandwidth, and then determine the set F in a greedy way, where T_i is computed by the TM algorithm;
Let $X = \{e \mid e \text{ is an overloaded edge}, e \in T_i, i = 0 \sim m-1\}$;
While ($X <> \Phi$) {// repair procedure
    Randomly pick an overloaded edge $e \in X$, $X = X - \{e\}$;
    Let W be a set of trees that contain the overloaded edge e;
    While (edge e is an overloaded edge){
        Find a tree T_j in set W to rebuild without edge e using the TM algorithm;
        If it fails to find such T_j, then stops; /* No valid solution is found. */
}}
Return F;
}

Figure 6. The greedy procedure

Tabu-search based heuristic procedure () {
Given a graph $G = (V, E)$, assume that D be a subset of V, $|D| = m$;
Let each node $i \in D$ initiate a multicast session, and be a root of a multicast routing tree;
For each node $i \in D$
If input bandwidth of node i < $(|D| - 1)$, then stops;
F = Greedy_procedure() ;
$F = \{T_0, T_1, ..., T_{m-1}\}$, and T_i is a multicast routing tree;
F is a valid initial solution;
I=0; useless_iters=5 ;
Initialize two circular queues Q and R, and let them be two tabu-lists;
Set the length of Q and R to be the minimum value of {10% of number of links, 50% of number of nodes};
While (I < max_iterations) {
Pick up a link e from edge-set E randomly such that $e \notin$ Q and e $\notin$ R;
Let $M = \{ T_j |\ T_j \in F$, and T_j contains the link $e \}$;
If $M <> \emptyset$, then store link e into tabu-list Q;
Save all the configurations of multicast routing trees in M;
Let $U = \emptyset$;
For each multicast tree T_j in M {
Delete the link e from T_j;
Based on the sub-tree with the root node, a new multicast tree is rebuilt using TM algorithm, and no link is overloaded;
If (the rebuilding process is successful)
$U = U \cup T_j$;
}
If ($U == \emptyset$)
Link e is a critical edge; store link e into tabu-list R;
Select a tree $T_k^{'}$ from U to be a new multicast tree for root node k, if the resulting total link cost is the smallest among all the trees in U;
Recover all the trees in M except for T_k back to their original configurations;
$F = F - \{T_k\} + \{T_k^{'}\}$;
If (the total link cost of F < the total link cost of global best solution) {
Let F to be the new global best solution;
[1] I=0;
}
Else count++ ;
If (count > useless_iters) {
F = the current global best solution;
count=0;
}
I++;
}

Figure 7. Tabu-search based heuristic procedure

5. Experimental Results

In this section, we have several sets of experiments on our tabu-search based algorithms in section 4 and the modified-MMST heuristic procedures in section 3 for solving the GMR problem. The purpose of this study is to compare the solution quality and executing performance between these algorithms. All the experiments of this study are done with the following experimental parameters: PIII 866 MHz CPU, 512MB RAM, Linux OS, and programs are developed by c++.

5.1. Random network generation

For the experiments conducted in this section, we have a procedure in this simulation for generating random directed graphs. First, we generate a random undirected graph based on the following equation:

$$P(\{u,v\}) = \quad \exp\frac{-d(u,v)}{L\sim}, \qquad [B]$$

Where p is the probability, $d(u, v)$ is the distance of two nodes, L is the maximum distance of any two nodes in the graph, ~ and are two positive and less than one numbers [2]. However, one problem with this model is that the number of degrees of each node is increased, as the number of nodes is increased. However, in real network, the number of degrees of each node tends to be a small value. Hence, a modified model reported in [7] is used to generate a random undirected graph with a small constant degree. The modification implemented is to scale $P(\{u,v\})$ by a factor $k*g/n$, where k is an empirical parameter, g is the mean degree and n is the number of nodes in the graph. In this study, the mean degree of nodes in the network is set to be 6.

After an undirected graph is created, we transform it to be a directed graph by making each undirected edge to be two directed links pointing two different end-nodes. A small number of nodes are randomly selected to be the source nodes. Each member of this source-node set must be a root of a multicast tree, where all the source nodes must be reachable from the root. Assume that the size of this source-node set is $|D|$; the mean bandwidth of the network is set to be $|D|*f$, where $f = 0.5 \sim 1.0$. The bandwidth $B(u,v)$ is then randomly set from $0.5*B_m$ to $1.5*B_m$, where B_m is the given mean bandwidth. It is possible that $B(u,v)$ may not be equal to $B(v,u)$. Nevertheless, the link cost of (u,v) should be the same as the link cost of (v,u), since it is set to be the Cartesian distance between u and v.

5.2. Lower bound study, tabu-search based procedure and modified-MMST procedure

In this part of experiments, a set of lower bounds and approximation solutions for the GMR problem is computed using MMST, modified-MMST procedure and tabu-based procedure respectively. Since the MMST procedure presented in Figure 4 is an exponential time algorithm, it is unlikely to test a network with hundreds nodes and links under the current experimental environment. As shown in Table 2, the largest benchmark used in this study is a network with 20 nodes and 94 links on the average. Although this benchmark is a small network, it is still infeasible for any exhaustive link-enumeration method.

For modified-MMST procedure tested in this experiment, modifications are made for it in order to accelerate the executing speed and save the memory space. Remember that in the section 3 each source node maintains a sorted list containing the minimum spanning trees and their neighbor trees. In this simulation, only first-order neighbor trees are derived and added in the list. In addition, only the first 6 least-cost spanning trees of all lists are kept for determining the approximation solution. For networks generated in this experiment, the probability to find a valid approximation solution with these 6-element lists is very high, unless there is no solution exists in the network. This modified approaches works because that the overall costs of all the spanning trees rooted at the source nodes should not spread over a large range of numbers, since the cost of a link is its Cartesian distance of two end nodes. Hence, a valid least-cost solution should be made by the spanning tress on the near top of lists.

Based on different number of source nodes, ten runs of experimental results on a 20-node network are given in Table 1. In Table 1, dist1 represents the distance between lower bounds and approximation solutions computed by modified-MMST procedure, whereas dist2 represents the distance between lower bounds and approximation solutions found by tabu-search based procedure. For about 50% of runs, the results with the 0% value of “dist1” indicate that the set of minimal spanning trees (the lower bound) found by MMST procedure does not contain any overloaded link. Hence, this set is the exact optimal solution. For a number of cases that “dist1” larger 0%, the exact optimal solution should be located on the point between the lower bound and the approximation solution found by modified-MMST procedure. Since the average values of “dist1” are 0.17% and 0.24% for the number of source nodes being 8 and 6, the solutions found by the modified-MMST is very close to the exact optimal solutions. Furthermore, the results that the “dist2” are 0.42% and 0.40% on the average show that the quality of solutions found by our tabu-search based algorithm is very high at this set of simulation.

In Table 2, several sets of benchmarks are designed to measure the quality of solutions found by the modified-MMST procedure and tabu-search based heuristic. Each data shown in Table 2 is an average value of ten runs. The largest number of source nodes is 8 because of the limit of memory space, while the mean bandwidth is set to be $1.0*|D|$ and $0.8*|D|$. In Table 2, "dev1" is used to measure distance between solutions found by tabu-search and modified-MMST procedures, and "dev2" is used to measure distance between lower bounds and approximation solutions found by tabu-search procedures. The optimal solution should be in the range from lower bound to approximation solution of modified-MMST. Since the average values of dev1 and dev2 are 0.21% and 0.78% respectively, it indicates that the relative errors between the optimal solutions and the approximation solutions found by our tabu-based heuristic should be less than 0.78% and greater than 0.21%.

For large-scale networks, it is more appropriate to solve the GMR problem with tabu-search based heuristic. A set of experiments of a 100-node network is shown in Table 3, where the data given in the 3rd column (greedy1) is the results from the greedy method by ignoring the bandwidth constraints, whereas the data given in "greedy2" column is the results after all the overloaded edges having been fixed by the repair procedure. The data given in the "tabu search" column is the results found by our heuristic algorithm. The last column (improvement ratio) is computed by the equation: (greedy2-tabu)/greedy2$*$100%. Every data shown in Table 1 is an average value over 10 executing results from the same simulation conditions. The improvement ratio is varied approximately from 4% to 2%. In general, for each group size |D|, improvement ratio is decreased as the mean bandwidth is increased. The reason is that the number of overloaded edges is inversely proportional to the bandwidth, and there is no much room left for our tabu search procedure to improve the solution quality.

6. Conclusions

In this paper, we present a method to establish the lower bound for the GMR problem. In addition, two heuristic methods, modified-MMST and tabu-search based procedures, are used to search for the approximation solutions. Based on the experiments conducted in this study, the approximation solutions found by modified-MMST method and tabu-search based heuristic are very close to optimal solutions for small-scale networks. The relative error is not greater than 0.78%. In addition, for large-scale network, the experimental results show that the tabu-search based heuristic is an effective procedure for the problem.

Table 1. Lower bound study for a 20-node network

- [a] number of nodes in the network = 20
- [a] lower bound is determined by MMST procedure
- [a] dist1=(modified-MMST – lower bound)/lower bound
- [a] dist2=(tabu search – lower bound)/lower bound
- [a] |D|: number of source nodes

#source nodes	mean bandwidth	lower bound	modified-MMST	tabu-search	dist1	dist2
8	8.1	3136	3140	3141	0.13%	0.16%
	8.3	2432	2432	2432	0.00%	0.00%
	7.9	3704	3704	3704	0.00%	0.00%
	8.1	4000	4005	4017	0.13%	0.43%
	7.9	3824	3870	3870	1.20%	1.20%
	7.6	3432	3434	3460	0.06%	0.82%
	7.8	3296	3302	3302	0.18%	0.18%
	8.1	3304	3304	3352	0.00%	1.45%
	7.7	3464	3464	3464	0.00%	0.00%
	7.7	3464	3464	3464	0.00%	0.00%
Average					0.17%	0.42%
6	5.9	2016	2016	2016	0.00%	0.00%
	6.2	2478	2498	2511	0.81%	1.33%
	5.9	1554	1554	1557	0.00%	0.19%
	6.0	2958	2958	2958	0.00%	0.00%
	5.9	2034	2034	2034	0.00%	0.00%
	6.1	1848	1861	1861	0.70%	0.70%
	5.9	2634	2644	2644	0.38%	0.38%
	6.0	2298	2312	2329	0.61%	1.35%
	6.0	2430	2430	2432	0.00%	0.08%
	6.2	2694	2694	2694	0.00%	0.00%
Average					0.25%	0.40%

Table 2. The solution quality comparison between lower bound, tabu-search procedure and modified-MMST procedure

[a] dev1=(tabu search – modified-MMST)/modified-MMST
[a] dev2=(tabu search – lower bound)/lower bound
[a] |D|: number of source nodes

#nodes (#mean edges)	|D|	mean bandwidth	lower bound	modified-MMST (cpu time)	tabu-search (cpu time)	dev1	dev2
20 (94)	8	5.9	3138.4	3160.6 (39.2 sec)	3173 (0.5 sec)	0.39%	1.1%
	8	7.9	3399.3	3409.1 (59.1sec)	3419.8 (0.5 sec)	0.31%	0.6%
	6	4.0	1942.2	1972.9 (25.4 sec)	1977.3 (0.3 sec)	0.22%	1.81%
	6	6.0	2294.4	2300.1 (32.8 sec)	2303.6 (0.3 sec)	0.15%	0.40%
	5	4.0	1443	1457.5 (17.3 sec)	1460.5 (0.2 sec)	0.21%	1.21%
	5	4.9	1683.5	1683.5 (87.6 sec)	1688.8 (0.2 sec)	0.31%	0.31%
	4	3.0	1217.2	1217.2 (50.4 sec)	1223.6 (0.2 sec)	0.53%	0.53%
	4	4.0	1058.0	1058.0 (42.5 sec)	1058.8 (0.2 sec)	0.08%	0.08%
15 (65)	7	5.0	2025.1	2053.0 (3.1 sec)	2052.6 (0.3 sec)	-0.02%	1.36%
	7	6.9	1722.7	1725.3 (3.1 sec)	1725.5 (0.3 sec)	0.01%	0.16%
	6	4.0	1266.0	1289.2 (1.0 sec)	1299.4 (0.1 sec)	0.79%	2.64%
	6	6.1	1470.6	1474.0 (0.9 sec)	1475 (0.2 sec)	0.07%	0.30%
	5	4.0	957.5	962.4 (0.9 sec)	964.7 (0.2 sec)	0.24%	0.75%
	5	5.0	1050.0	1050.0 (0.6 sec)	1052.2 (0.2 sec)	0.21%	0.21%
	4	3.0	726.4	726.4 (0.9 sec)	728.1 (0.1 sec)	0.23%	0.23%
	4	4.0	648.4	648.4 (0.9 sec)	650.1 (0 sec)	0.26%	0.26%
10 (30)	7	5.0	1206.1	1241.9 (2.5 sec)	1242 (0.1 sec)	0.01%	2.98%
	7	7.1	1188.6	1191.5 (2.5 sec)	1192.1 (0.1 sec)	0.05%	0.29%
	6	4.0	868.2	888.7 (0.5sec)	891.4 (0.1 sec)	0.30%	2.67%
	6	6.0	923.4	923.4 (0.4 sec)	924 (0 sec)	0.06%	0.06%
	5	4.0	775.5	779.0 (0.1 sec)	779.1 (0 sec)	0.01%	0.46%
	5	5.0	827.0	827.0 (0.1 sec)	828.3 (0 sec)	0.16%	0.16%
	4	3.0	405.2	405.2 (0.1 sec)	405.2 (0 sec)	0.00%	0.00%
	4	4.0	418.0	418.0 (0.1 sec)	418.5 (0 sec)	0.12%	0.12%

The average value of dev1 = 0.21%
The average value of dev2 = 0.78%

Table 3. The experimental results for tabu-search based heuristic

- [a] Number of nodes in the network =100.
- [a] |D|= number of source nodes.
- [a] The greedy1 is found by the greedy method based on the assumption that all the links are with infinite bandwidth.
- [a] The greedy2 derived from greedy1 contains no overloaded link.
- [a] dev1= (greedy2 – tabu search)/greedy2.

|D|	mean bandwidth	mean number of overloaded links	greedy1	greedy2	tabu search	dev1 (100%)	cpu (sec)
10	4.98	17.1	36737	41144	39520	4.0%	10.92
10	5.94	14.7	33915	38093	36685	3.7%	12.66
10	7.96	8.0	34212	36895	35711	3.2%	7.96
10	9.99	2.8	33034	33730	32509	3.6%	9.99
15	8.93	20.9	70784	80171	76733	4.3%	33.7
15	9.95	19.0	70262	79081	75981	3.9%	30.5
15	11.92	10.4	69609	73927	72169	2.4%	26.0
15	13.0	8.5	67749	71069	68855	3.1%	28.3
15	15.0	6.1	73059	75356	72940	3.2%	43.5
20	9.96	29.3	105773	125160	118877	5.0%	63.5
20	12.98	27.2	121913	138785	132984	4.2%	72.4
20	15.94	17.6	114387	124442	120231	3.4%	82.1
20	16.98	15.1	113766	121198	117279	3.2%	81.0
20	20.05	9.1	114282	119287	117152	1.8%	60.7
25	11.89	37.1	152544	190561	181898	4.6%	123.5
25	14.81	28.8	146125	176441	168913	4.3%	124.1
25	16.92	24.7	170637	190559	183278	3.8%	122.6
25	19.94	21.0	160275	182003	176325	3.1%	133.8
25	21.78	18.5	163125	181738	175092	3.7%	139.1
25	24.70	15	169243	183522	177623	3.2%	139.9
30	14.83	41.8	211315	265494	254566	4.1%	209.7
30	17.80	34.9	236220	282181	270210	4.2%	242.9
30	20.00	31.2	215030	249616	241999	3.1%	197.8
30	23.60	25.5	223004	251914	243100	3.5%	224.6
30	25.90	22.0	220846	242387	234278	3.4%	270.6
30	29.70	18.0	222239	241276	235675	2.3%	229.1

Reference

[1] Anees Shaikh and Kang Shin, "Destination-Driven Routing for Low-Cost Multicast," *IEEE Journal on Selected Areas in Communications,* vol. 15, No. 3, pp.373-381, 1997.

[2] B. M. Waxman, "Routing of multipoint connections," *IEEE Journal on Selected Areas in Communications*, vol. 6, no. 9, pp. 1617-1622, 1988.

[3] H. Takahashi and A. Matsuyama, "An approximation solution for the Steiner tree problem in graphs," Mathematica Japonica, vol. 24, no. 6, pp. 573-577, 1980.

[4] L. Kou, G. Markowsky, and L. Berman, "A fast algorithm for Steiner trees in graphs," *Acta Informatica*, vol. 15, pp.141-145, 1981.

[5] Matthew Doar and Ian Leslie, "How Bad is Naïve Multicast Routing," INFOCOM, pp. 82-89 1993.

[6] S.K. Lodha, D. Bhatia, **"**Bipartitioning circuits using TABU search," pp. 223-227, *Proceedings of ASIC Conference* 1998.

[7] Khaled S. AL-SULTAN, "A Tabu Search Approach to The Clustering Problem," *Pattern Recognition*, vol. 28, No. 9, pp. 1443-1451, 1999.

[8] S.M. Sait, H. Youssef, M.M. Zahra, **"**Tabu search based circuit optimization," *Proceedings of the 8th Great Lakes Symposium on VLSI*, 1998.

[9] Xiaohua Jia and Lusheng Wang, "A group multicast routing algorithm by using multiple minimum Steiner trees," *Computer Communications*, vol. 20, pp. 750-758, 1997.

[10] C.P. Low and N. Wang, "An efficient algorithm for group multicast routing with bandwidth reservation," *Computer Communication*, vol. 23, pp. 1740-1746, 2000.

[11] V.J. Rayward-Smith, A. Clare, "On finding Steiner vertices," Networks, 16(3), pp. 283-294, 1986.

[12] Kevin C. Almeroth, and Mostafa H. Ammar, "The Use of Multicast Delivery to Provide a Scalable and Interactive Video-on-Demand Service", *IEEE Journal on Selected Areas in Communications*, vol. 14, no. 6, pp. 1110-1122, August 1996.

[13] Sanjeev Verma, Rajesh K. Pankaj, and Alberto Leon-Garica, "QoS based multicast routing algorithms for real time applications", *Performance Evaluation*, vol. 34 pp. 273-294, 1998.

[14] Dinkar Sitaram and Asit Dan, *Multimedia Servers*, Morgan Kaufmann Publishers, 2000.

[15] Fred Glover, Manuel Laguna, *Tabu search* , Kluwer Academic Publishers, 1997.